

树状数组

Hex



树状数组 Fenwick Tree

基础介绍



用于高效维护前缀和



支持单点修改、区间查询

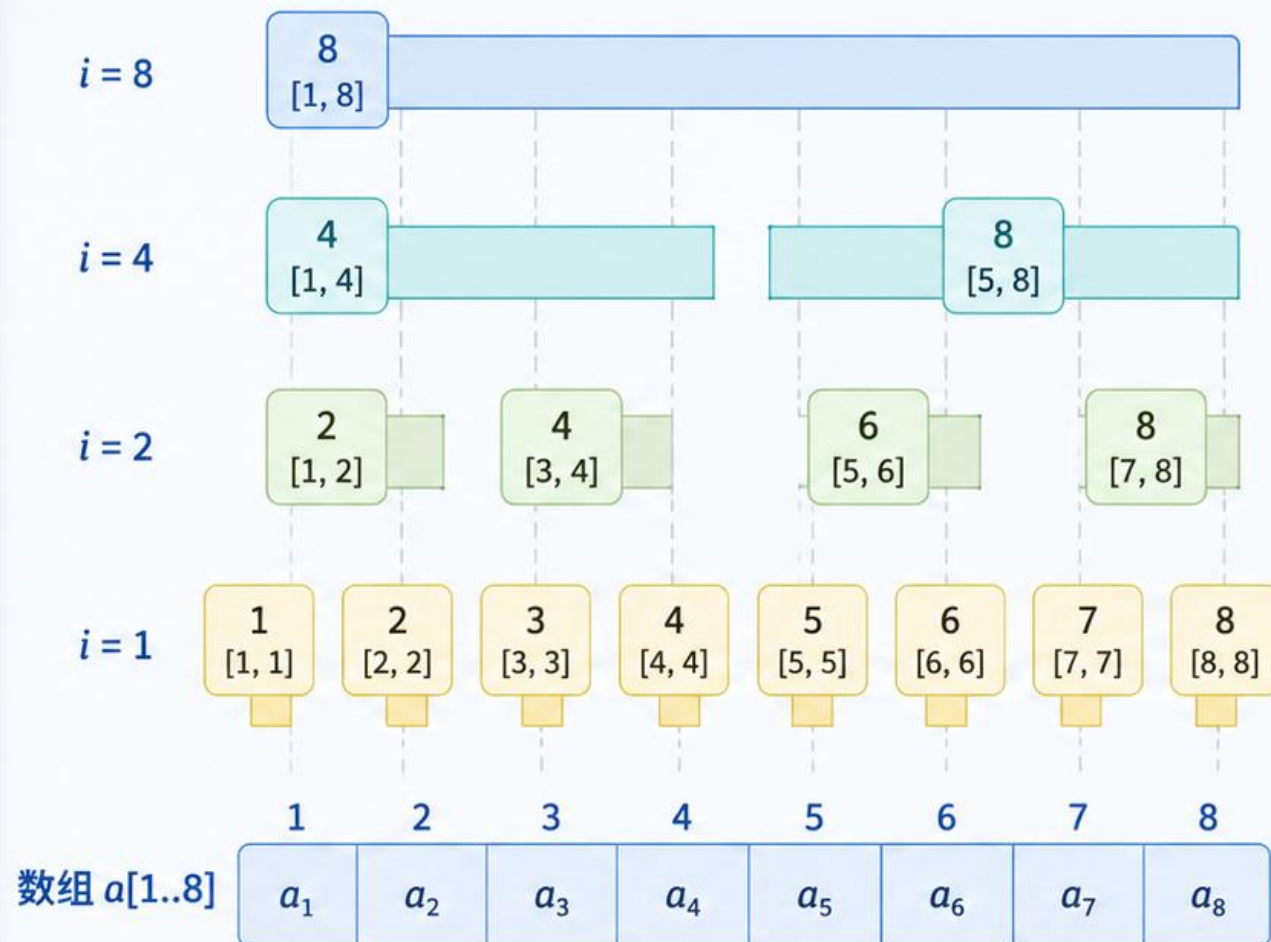


时间复杂度: $O(\log n)$



空间复杂度: $O(n)$

树状数组各节点覆盖的区间 (示意)





为什么需要树状数组？

问题背景

操作	普通数组	前缀和数组
单点修改	$O(1)$	$O(n)$
区间求和	$O(n)$	$O(1)$



当前缀和数组遇到频繁修改时，维护成本很高。



直观理解：一次修改影响很多前缀和

假设数组 a 与其前缀和数组 s 如下：





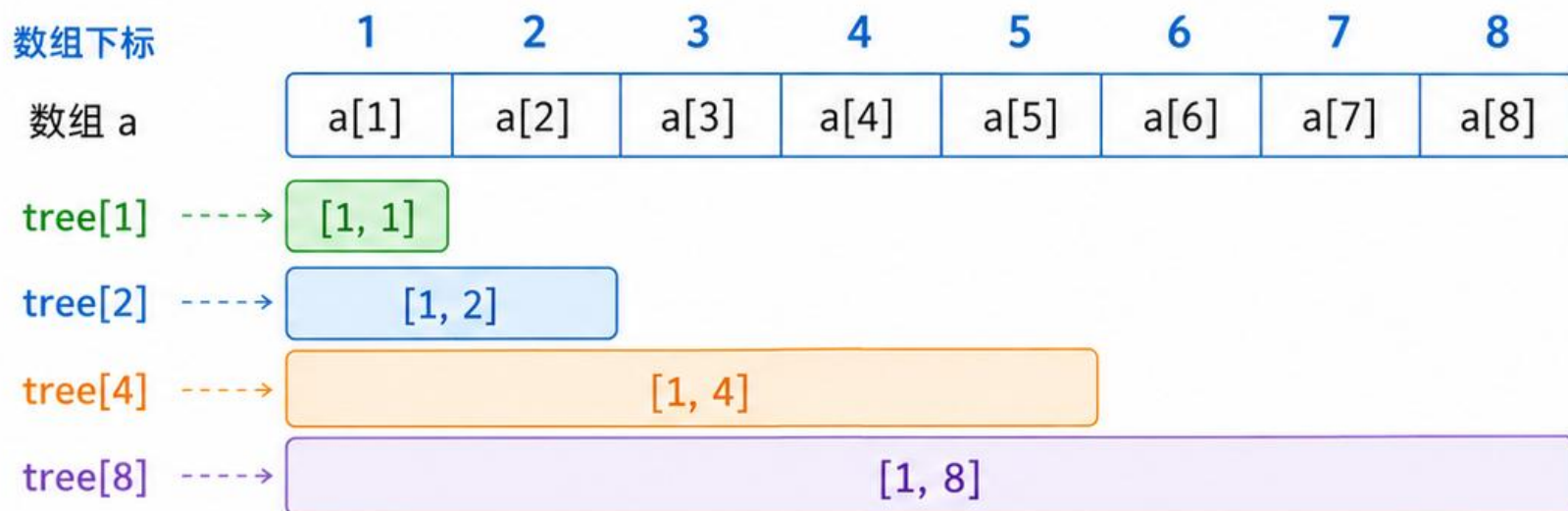
树状数组的核心思想

用多个“局部前缀和”拼出答案

- $tree[i]$ 不直接等于 $a[i]$
- $tree[i]$ 保存一段连续区间的和
- 查询前缀和时，把若干段拼起来



$tree[i]$ 保存区间
 $[i - \text{lowbit}(i) + 1, i]$ 的和



i 说明

每个 $tree[i]$ 覆盖的都是以 i 结尾的连续区间，长度为 $\text{lowbit}(i)$ 。



核心函数

lowbit(x) = $x \& -x$

可视化说明：突出最低位的1

12 1 1 0 0 → 最低位的 1
 ↑
 对应的值：4



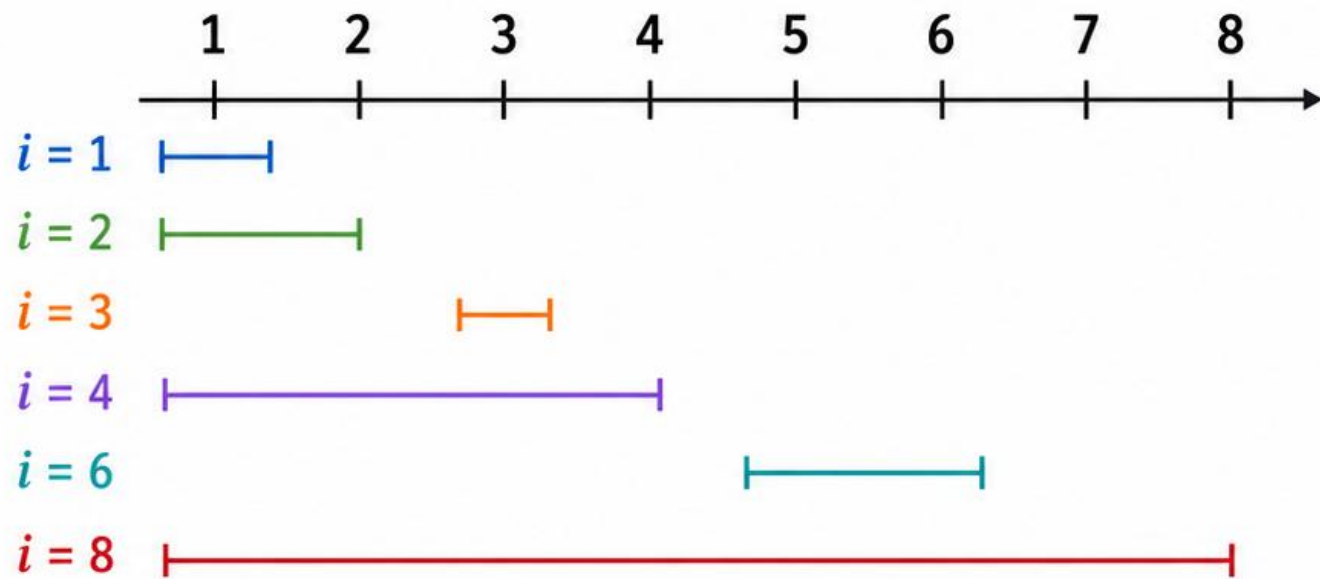


tree[i] 的管辖区间



每个结点维护哪一段和？

$$\text{tree}[i] = \text{sum}(a[i - \text{lowbit}(i) + 1] \dots a[i])$$



i	$\text{lowbit}(i)$	$\text{tree}[i]$ 覆盖区间
1	1	[1, 1]
2	2	[1, 2]
3	1	[3, 3]
4	4	[1, 4]
6	2	[5, 6]
8	8	[1, 8]



单点修改：一路向上更新

update / add 操作



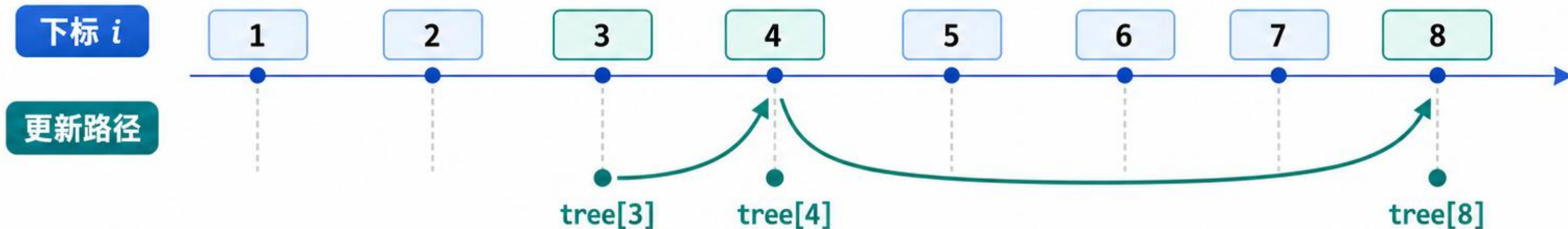
当执行 $a[x] += k$ 时，位置 x 的值增加了 k 。
因此，所有覆盖到位置 x 的树状数组结点 $tree[i]$ 都需要增加 k ，才能保持前缀和的正确性。

C++ 代码

```
void add(int x, int k) {  
    while (x <= n) {  
        tree[x] += k;  
        x += lowbit(x);  
    }  
}
```



示例： $\text{add}(3, +5)$ 会更新： $\text{tree}[3] \rightarrow \text{tree}[4] \rightarrow \text{tree}[8]$





区间和查询

两个前缀和相减

$$\text{sum}(l, r) = \text{sum}(r) - \text{sum}(l - 1)$$

例：

$$\text{sum}(3, 7) = \text{sum}(7) - \text{sum}(2)$$

可视化解释



区间 $[1, 7]$

-

区间 $[1, 2]$

=

区间 $[3, 7]$



树状数组本质上高效支持前缀和，因此区间和可以转化为两个前缀和。



总结与竞赛应用

• 什么时候使用树状数组？ •



适合解决



- 单点修改 + 区间查询



- 区间修改 + 单点查询



- 逆序对统计



- 动态前缀和



- 离散化后维护频率



不太适合



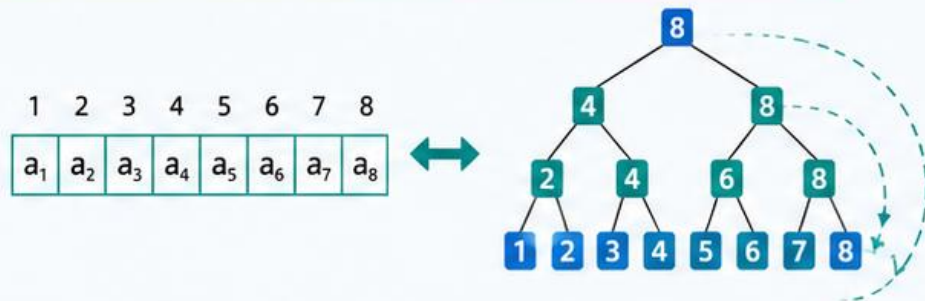
- 区间最值的复杂修改



- 需要维护复杂区间信息的问题



树状数组 = 用二进制思想维护前缀信息的高效数据结构



P3374 [模板] 树状数组 1

如题，已知一个数列，你需要进行下面两种操作：

1. 将某一个数加上 x ；
2. 求出某区间每一个数的和。

输入输出样例

输入 #1

复制

```
5 5
1 5 4 2 3
1 1 3
2 2 5
1 3 -1
1 4 2
2 1 4
```

输出 #1

复制

```
14
16
```

P3374 [模板] 树状数组 1

```
const int Maxn = 5e5+10;
int n, m;
int c[Maxn];
// 找 二进制下 最低为 1
int lowbit(int x){
    return x&(-x);
}
// 把 x 位置加上 v
void Add(int x, int v){
    while( x <= n ){
        c[x] += v;
        x += lowbit(x);
    }
}
// 求 1~x 的前缀和
int Sum(int x){
    int sum = 0;
    while( x > 0 ){
        sum += c[x];
        x -= lowbit(x);
    }
    return sum;
}
```

```
int main(){
    scanf("%d %d", &n, &m);
    for(int i=1,x;i<=n;i++){
        scanf("%d", &x);
        Add(i, x);
    }
    for(int i=1;i<=m;i++){
        int op, x, y;
        scanf("%d %d %d", &op, &x, &y);
        if( op == 1 ){
            Add(x, y);
        }
        else {
            printf("%d\n", Sum(y) - Sum(x-1));
        }
    }
    return 0;
}
```


P3368 [模板] 树状数组 2

如题，已知一个数列，你需要进行下面两种操作：

1. 将某区间每一个数加上 x ；
2. 求出某一个数的值。

输入 #1

复制

输出 #1

复制

```
5 5
1 5 4 2 3
1 2 4 2
2 3
1 1 5 -1
1 3 5 7
2 4
```

```
6
10
```

P3368 [模板] 树状数组 2

```
int main(){
    scanf("%d %d", &n, &m);
    for(int i=1;i<=n;i++){
        scanf("%d", &a[i]);
        Add(i, a[i]-a[i-1]);
    }
    for(int i=1;i<=m;i++){
        int op, x, y, k;
        scanf("%d %d", &op, &x);
        if( op == 1 ){
            scanf("%d %d", &y, &k);
            Add(x, k);
            Add(y+1, -k);
        }
        else {
            printf("%d\n", Sum(x));
        }
    }
    return 0;
}
```

P2184 贪婪大陆

小 FF 最后一道防线是一条长度为 n 的战壕，小 FF 拥有无数多种地雷，而 SCV 每次可以在 $[L,R]$ 区间埋放同一种不同于之前已经埋放的地雷。由于情况已经十万火急，小 FF 在某些时候可能会询问你在 $[L',R']$ 区间内有多少种不同的地雷，他希望你能尽快的给予答复。

$$0 \leq n, m \leq 10^5$$

输入 #1

```
5 4
1 1 3
2 2 5
1 2 4
2 3 5
```

复制

输出 #1

```
1
2
```

复制

P2184 贪婪大陆 题解

• 两个树状数组维护区间左右端点 •

1 题意提炼

- 操作 1: 加入一种新地雷, 覆盖区间 $[l, r]$;
- 操作 2: 询问当前在 $[L, R]$ 内出现了多少种不同的地雷。



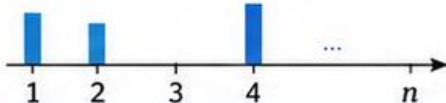
每一种地雷只对应一次加入的区间,
因此询问答案 = 与 $[L, R]$ 相交的已加入区间数量

3 数据结构设计

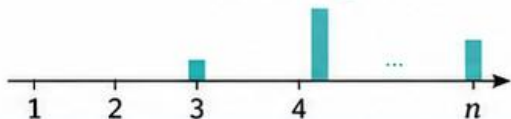
- BIT_L : 统计每个位置作为左端点的出现次数
- BIT_R : 统计每个位置作为右端点的出现次数

操作 $q = 1$ (加入 $[l, r]$): $\text{add}(\text{BIT}_L, l, 1), \text{add}(\text{BIT}_R, r, 1)$
操作 $q = 2$ (查询 $[l, r]$): $\text{ans} = \text{sum}(\text{BIT}_L, r) - \text{sum}(\text{BIT}_R, l - 1)$

左端点统计 (BIT_L)



右端点统计 (BIT_R)



2 相交判定

对于已加入的区间 $[l, r]$, 它与查询区间 $[L, R]$ 相交, 当且仅当

$$l \leq R \text{ 且 } r \geq L$$

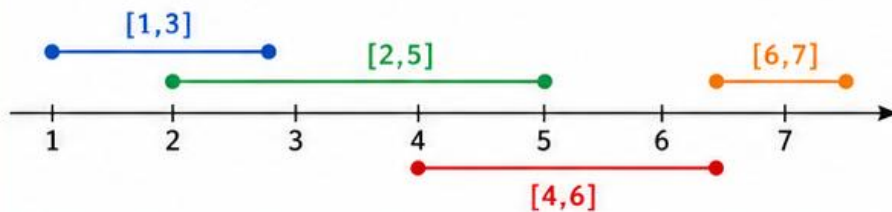


答案 = 已加入区间中左端点 $\leq R$ 的个数 - 右端点 $< L$ 的个数

$$\text{ans} = \text{prefixLeft}(R) - \text{prefixRight}(L - 1)$$

4 小例子

已加入区间: $[2, 5]$, $[1, 3]$, $[6, 7]$ 查询区间: $[4, 6]$



- $[2, 5]$ 与 $[4, 6]$ 相交 ✓
- $[1, 3]$ 与 $[4, 6]$ 不相交 ✗
- $[6, 7]$ 与 $[4, 6]$ 相交 ✓

因此, 答案 = 2

用公式验证: $\text{prefixLeft}(6) = 3$, $\text{prefixRight}(3) = 1$, 所以 $\text{ans} = 3 - 1 = 2$

扩展

如果在放雷时给出雷的种类可以分为多种情况：

- 同一种雷只保留最新一次放置

一次操作： $1\ c\ l\ r$ 在区间 $[l, r]$ 中放置种类为 c 的雷。之前放置的种类为 c 的雷失效。

每次更新前把之前有的操作回撤，再插入新的

- 同一种雷可以多次放置，并且都有效。

(a) 雷的种类较少 **BITSET**

(b) 先放雷再查询 将查询按右端点排序，在遍历查询时，逐渐放雷。

(c) 完全在线 非常麻烦

P1972 [SDOI2009] HH 的项链

HH 有一串由各种漂亮的贝壳组成的项链。HH 相信不同的贝壳会带来好运，所以每次散步完后，他都会随意取出一段贝壳，思考它们所表达的含义。HH 不断地收集新的贝壳，因此，他的项链变得越来越长。

有一天，他突然提出了一个问题：某一段贝壳中，包含了多少种不同的贝壳？这个问题很难回答.....因为项链实在是太长了。于是，他只好求助睿智的你，来解决这个问题。

输入 #1

```
6
1 2 3 4 3 5
3
1 2
3 5
2 6
```

复制

输出 #1

```
2
2
4
```

复制

P1972 [SDOI2009] HH 的项链 题解

离线查询 + 树状数组维护每种颜色的最后一次出现

1 题意提炼

给定长度为 n 的项链数组 $a[1..n]$ ，多次询问区间 $[l, r]$ 中有多少种不同的贝壳。

★ 静态数组，多次区间不同数查询 —— 经典离线题

3 为什么正确



对于每种颜色，在前缀 $[1, r]$ 中只保留最后一次出现的位置。若该最后一次出现位置落在 $[l, r]$ 内，则说明这种颜色在区间 $[l, r]$ 中出现过；反之没有出现。因此，树状数组中 $[l, r]$ 内的标记个数，恰好等于区间 $[l, r]$ 中不同值的种类数。

2 核心思路

将所有询问按右端点 r 从小到大排序；扫描 i 从 1 到 n 。维护每种颜色的最后一次出现位置 $last[color]$ 。在树状数组中，仅把每个值“最新出现”的位置标记为 1，若之前存在更早的最新位置，则在该位置删除标记。这样，当扫描到位置 r 时，任意询问 $[l, r]$ 的答案就是树状数组中落在区间 $[l, r]$ 内的标记个数。

1 if $last[a[i]]$ 存在, $add(last[a[i]], -1)$

2 $add(i, +1)$

3 $last[a[i]] = i$

4 $answer([l, r]) = sum(r) - sum(l - 1)$

4 小例子

数组 $a = [1, 2, 1, 3, 2]$ 。

扫描到 $i = 4$ 时，最新出现位置为：

颜色 1 在 3，颜色 2 在 2，颜色 3 在 4，因此树状数组在位置 2, 3, 4 处标记为 1。

对于询问 $[2, 4]$ ，答案 = 3，因为区间内不同的值为 $\{1, 2, 3\}$ 。

下标 i	1	2	3	4	5
$a[i]$	1	2	1	3	2

BIT 标记 (扫描到 $i = 4$)	0	1	1	1	0
		↑ 2	↑ 3	↑ 4	